Architecture 1001: x86-64 Assembly
Writing Assembly

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

Writing Assembly 1

- There are times when you actually have to code asm in order to do something from a C/C++ file
  - Fairly common in firmware, OS, or driver programming. Because there are many aspects of hardware which can only be accessed with special instructions
  - If you were making a crypto library, you would want access to hardware cryptographic acceleration instructions
  - Or maybe you just want full control over the code being generated for optimization purposes

Writing Assembly 2

- Being able to write assembly is a great way to simply experiment with instructions we have, and haven't covered in this class
  - Though getting the syntax right for the desired instructions/assemblers is often annoying

Inline Assembly

- Inline assembly is a way to include assembly directly in-place in a C/C++ file
  - GCC supports inline assembly in GNU Assembler (gas) syntax
  - VisualStudio supports inline assembly for 32-bit binaries, but discontinued support for 64-bit

Standalone Assembly 1

- Pure assembly can be written in a standalone file, assembled by an assembler into an object file, and then linked into the final binary with other compiled objects
- This is the required path for raw x86-64 assembly in Visual Studio

Standalone Assembly 2

- Different assemblers have their own syntax variants and helper mechanisms to make programming in assembly less painful for those who must
  - MASM (Microsoft Assembler) is what's used with Visual Studio
  - NASM (Netwide Assembler) is cross-platform

Raw Bytes

- Most inline/standalone assembly-writing solutions also have support for emitting raw bytes, which the processor will interpret as instructions
- This can be a good way to check your understanding of how you think the manual is specifying a particular assembly instruction is encoded, vs. what a disassembler ultimately says you encoded
- This can also be used to work around not knowing the correct syntax for some instruction in some assembler